Projet POO C++ — Moteur masses-ressorts

13 octobre 2026

Objectif

Concevoir en C++ un moteur de simulation physique 2D à base de particules reliées par des ressorts, qui anime en temps réel trois scènes : une corde, un tissu et un objet mou.

Le moteur doit être extensible : ajouter une nouvelle force, un nouvel intégrateur ou un nouveau mode d'affichage doit se faire en écrivant une nouvelle classe, sans modifier le code existant du moteur.

En fin de projet, l'équipe livre :

- un programme qui ouvre une fenêtre, affiche les trois scènes et permet d'attraper une particule à la souris ;
- un code organisé en bibliothèque physique indépendante de l'affichage ;
- un rapport court qui relie chaque notion du cours à un endroit précis du code ;
- une démonstration en soutenance.

Contraintes

Contrainte	Valeur imposée
Équipe	3 personnes
Charge de travail	6 h minimum, 12 h maximum par personne
Langage	C++17
Compilation	CMake, options `-Wall -Wextra -Werror`, zéro avertissement
Affichage	SFML 2.6 si le professeur l'autorise, sinon sortie SVG image par image (comme au TP 2)
Bibliothèques interdites	Toute bibliothèque de physique ou de mathématiques (Box2D, Eigen, glm…)
Mémoire	Aucune fuite à la sortie, vérifié avec `valgrind` ou `-fsanitize=address`
Versionnement	Un dépôt Git commun, un tag Git par jalon (`J1`, `J2`, `J3`, `J4`)

Un jalon est considéré comme livré quand son tag existe et que tous ses critères de validation sont cochés. Les éléments marqués « bonus » ne sont pas exigés.

Si le groupe ne dispose que de 6 h, le socle minimal est : jalons 1 et 2 complets, jalon 3 avec rendu SVG seulement, rapport du jalon 4 sans extension.

Rappels de physique

La physique se limite à quatre formules et à une boucle à pas de temps fixe. Les unités sont celles du SI (mètres, secondes, kilogrammes) ; la conversion en pixels est le travail du rendu, jamais du moteur.

Gravité. Chaque particule de masse m subit son poids, avec $g = 9{,}81\ \mathrm{m/s^2}$ vers le bas.

$$\vec{F}_g = m\,\vec{g}$$

Ressort amorti (Hooke). Entre deux particules A et B, avec $\vec{d} = \vec{x}_B - \vec{x}_A$, une longueur au repos L_0, une raideur k et un amortissement c :

$$\vec{F}_A = \left[\, k\left(\ \|\ \vec{d}\ \|\ - L_0\right) + c\left(\vec{v}_B - \vec{v}_A\right)\cdot \widehat{d}\,\right]\widehat{d}, \qquad \vec{F}_B = -\vec{F}_A$$

Frottement de l'air. Une force opposée à la vitesse, de coefficient b :

$$\vec{F}_{air} = -\,b\,\vec{v}$$

Intégrateurs. Deux méthodes à implémenter, avec $\vec{a} = \vec{F}/m$ et un pas dt :

- Euler semi-implicite : on met d'abord à jour la vitesse, puis la position avec la nouvelle vitesse.
- Verlet : on stocke la position précédente et la vitesse est déduite des positions.

$$\text{Euler} : \vec{v} \leftarrow \vec{v} + \vec{a}\,dt, \quad \vec{x} \leftarrow \vec{x} + \vec{v}\,dt$$

$$\text{Verlet} : \vec{x}_{n+1} = 2\vec{x}_n - \vec{x}_{n-1} + \vec{a}\,dt^2$$

Boucle de simulation. Pas fixe $dt = 1/120$ s ; si l'affichage tourne à 60 images/s, on fait 2 pas de physique par image. Chaque pas : remise à zéro des forces, application de toutes les forces, intégration, puis contraintes.

Contraintes.

- Particule épinglée : sa masse inverse vaut 0, elle ne bouge plus.
- Sol : si une particule passe sous le sol, on la replace dessus et on inverse sa vitesse verticale multipliée par un coefficient de restitution e ($0 \le e \le 1$).

Architecture imposée

Toutes les entités du moteur partagent une seule base `Entity` par héritage virtuel, et `World` ne manipule que des interfaces. Les noms de classes ci-dessous sont imposés ; l'équipe peut ajouter des classes, pas en retirer.

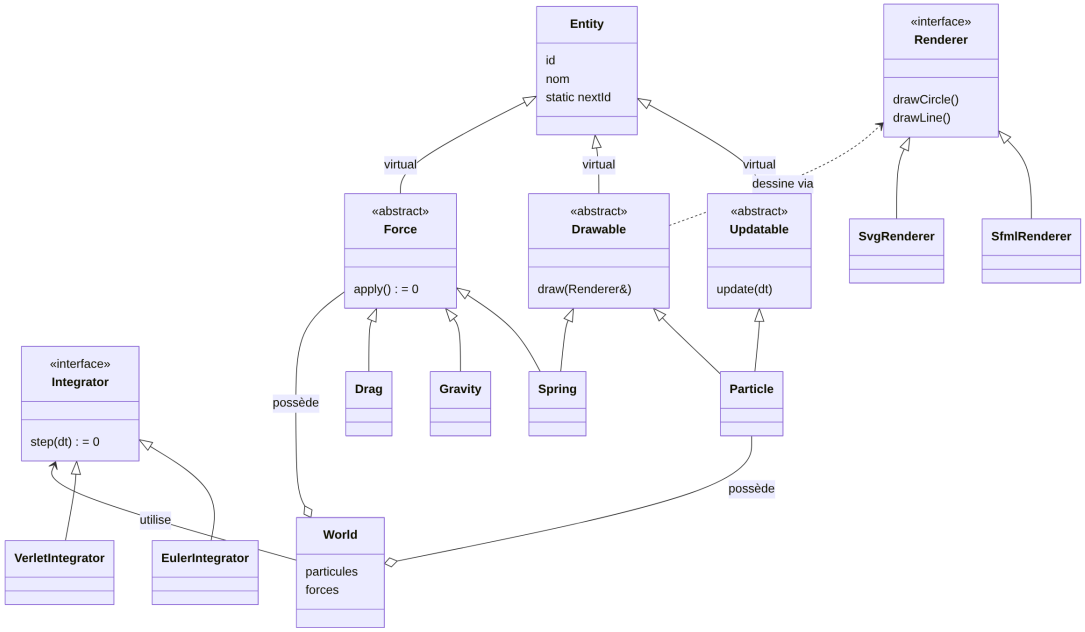


Diagramme de classes minimal

Une `Particle` est à la fois mise à jour et dessinée ; un `Spring` est à la fois une force et un élément dessiné. Sans héritage virtuel, chacune contiendrait deux `Entity` et donc deux identifiants. L'interface `Renderer` est déclarée dans la bibliothèque physique, ses implémentations vivent dans `render/`.

Arborescence du dépôt

```
projet-ressorts/
├── CMakeLists.txt
├── README.md
├── physics/            bibliothèque statique, aucun en-tête SFML
│   ├── Vec2, Entity, Particle, World
│   ├── forces/         Force, Gravity, Spring, Drag
│   ├── integrators/    Integrator, Euler, Verlet
│   └── Renderer.hpp    interface seule
├── render/             SfmlRenderer, SvgRenderer
├── scenes/             pendule, corde, tissu, blob
├── app/main.cpp
├── tests/
└── rapport/
```

Notions du cours à mobiliser

Chaque ligne est obligatoire, sauf mention « bonus ». Le rapport final reprend ce tableau en ajoutant pour chaque ligne le fichier et la ligne de code correspondants.

Notion du cours	Où elle apparaît dans le projet	Jalon
Surcharge d’opérateurs	`Vec2` : `+ - * / += -= == <<`, en méthodes membres et fonctions libres	J1
`const` et références	Passage des `Vec2` par `const&`, méthodes de lecture `const` partout	J1
`namespace`	`phys::` pour le moteur, `render::` pour l’affichage	J1
Liste d’initialisation	Tous les constructeurs, y compris les membres `const` (`Spring::restLength`)	J1
Classe abstraite, `virtual`, `override`	`Force::apply()`, `Integrator::step()`	J2
Destructeur virtuel	Toute classe de base polymorphe	J2
Upcasting et liaison dynamique	`World` ne manipule que des `Force*` et un `Integrator&`	J2
Règle des trois et `new`/`delete`	`World` possède ses particules et ses forces allouées sur le tas	J2
`enum class`	Type d’intégrateur, mode de rendu, état de la simulation	J2
Lambdas	Construction des scènes, recherche de la particule la plus proche de la souris	J2
Héritage multiple d’interfaces	`Particle : Updatable, Drawable` et `Spring : Force, Drawable`	J3
Héritage virtuel (diamant)	`Updatable` et `Drawable` héritent virtuellement d’`Entity`	J3
`static` dans une classe	`Entity::nextId`, compteur d’identifiants partagé	J3
`dynamic_cast`	Sélection à la souris : retrouver une `Particle` parmi des `Entity*`	J3
Bibliothèque statique (TP 2)	`physics` compilée en `.a`, sans aucun en-tête SFML	J3
Templates (bonus)	`Vec2<T>` ou un conteneur générique d’entités	J4
Exceptions (bonus)	Chargement de scène depuis un fichier texte invalide	J4

Jalon 1 — Fondations (environ 2 h)

But. Une particule tombe sous la gravité et rebondit sur le sol, et le résultat est vérifié par des tests.

À produire

- Le squelette du dépôt et un `CMakeLists.txt` qui compile la bibliothèque `physics`, le programme et les tests.
- La classe `phys::Vec2` avec ses opérateurs, `length()`, `normalized()` et `dot()`.
- La classe `phys::Particle` : position, vitesse, masse inverse, accumulateur de forces, état épinglé.
- Une boucle minimale codée en dur : gravité, Euler semi-implicite, rebond sur le sol.
- Une sortie texte de la position à chaque pas, ou un fichier CSV `t, x, y`.

Critères de validation

- ☐ `cmake .. && make` compile sans aucun avertissement avec `-Wall -Wextra -Werror`.
- ☐ Les tests de `Vec2` passent : au moins un test par opérateur, plus `length()` et `normalized()`.
- ☐ Une particule lâchée sans vitesse à 10 m du sol l'atteint à $t = 1,43$ s, à 2 % près (valeur théorique : $\sqrt{2h/g}$).
- ☐ Avec $e = 0,5$, la hauteur du premier rebond vaut environ 2,5 m ($e^2 \times 10$ m), à 5 % près.
- ☐ Une particule épinglée reste immobile pendant 10 s de simulation.
- ☐ Le `README` explique comment compiler et lancer le programme et les tests.

Tag Git : `J1`.

Jalon 2 — Forces, ressorts et polymorphisme (environ 3 h)

But. Le moteur simule une corde et un pendule à partir de forces et d'intégrateurs interchangeables, sans rien savoir de leurs types concrets.

À produire

- La classe abstraite `Force` et trois forces concrètes : `Gravity`, `Spring`, `Drag`.
- L'interface `Integrator` et deux implémentations : `EulerIntegrator`, `VerletIntegrator`.
- La classe `World` : elle possède les particules et les forces, et exécute la boucle de simulation.
- Deux scènes construites par des fonctions : un pendule (une particule épinglée, une libre, un ressort raide) et une corde de 20 particules.
- Une fonction qui calcule l'énergie totale du système (cinétique + potentielle de pesanteur + élastique) et l'écrit dans un CSV.

Critères de validation

- ☐ `World` ne contient aucun `if` ni `switch` sur le type d'une force ou d'un intégrateur.
- ☐ Ajouter une force (par exemple un vent constant) se fait en créant une seule nouvelle classe, sans modifier `World`.
- ☐ L'intégrateur peut être changé pendant l'exécution, via un `enum class` et une méthode `World::setIntegrator()`. Le seul `switch` autorisé est dans une fonction fabrique `makeIntegrator()`, hors de `World`.

- [ ] Sans amortissement, l'énergie du pendule avec Verlet varie de moins de 5 % sur 10 s. Un graphique compare Euler et Verlet.
- [ ] `World` respecte la règle des trois : copie profonde, ou copie interdite avec `= delete`, choix justifié en commentaire.
- [ ] `valgrind` ne signale aucune fuite à la fin d'une simulation de la corde.

Tag Git : `J2`.

Jalon 3 — Affichage temps réel et héritage multiple (environ 4 h)

But. Les quatre scènes (pendule, corde, tissu, blob) s'affichent en temps réel, l'utilisateur peut attraper une particule à la souris, et le moteur reste compilable sans la bibliothèque d'affichage.

À produire

- La hiérarchie en diamant : `Entity` (identifiant et nom), puis `Updatable` et `Drawable` qui en héritent virtuellement.
- L'interface `Renderer` (`drawCircle`, `drawLine`, `clear`, `display`) et deux implémentations : `SfmlRenderer` et `SvgRenderer`.
- Une troisième scène, le tissu : une grille de 20 × 20 particules épinglée par ses deux coins du haut, avec ressorts structurels et diagonaux.
- Une quatrième scène, le blob : un anneau de particules relié par des ressorts de bord et des ressorts internes, qui rebondit sur le sol.
- Les interactions clavier et souris : touches 1 à 4 pour changer de scène, I pour changer d'intégrateur, Espace pour mettre en pause, clic-glisser pour déplacer une particule.

Critères de validation

- [ ] Une `Particle` ne contient qu'un seul sous-objet `Entity` : `particle.getId()` compile sans qualification.
- [ ] Le rapport donne `sizeof(Particle)` avec et sans héritage virtuel et explique la différence.
- [ ] La bibliothèque `physics` est compilée en `.a` et aucun de ses fichiers n'inclut d'en-tête SFML (vérifiable avec `grep`).
- [ ] Passer du rendu SFML au rendu SVG ne demande de modifier que `main.cpp`.
- [ ] Le tissu de 400 particules s'affiche à 60 images/s ou plus sur une machine de TP, en compilation `-O2`.
- [ ] La particule attrapée est retrouvée par une lambda (la plus proche du clic) puis un `dynamic_cast` depuis `Entity*`.

Tag Git : `J3`.

Jalon 4 — Extension, rapport et soutenance (environ 2 h)

But. L'équipe prouve que l'architecture est extensible en ajoutant une fonctionnalité, puis défend ses choix de conception.

Une extension au choix (obligatoire)

- Déchirure du tissu : un ressort étiré au-delà de 2 fois sa longueur au repos se casse et disparaît.

- Vent variable : une force dont l'intensité suit une fonction du temps passée sous forme de lambda (`std::function`).
- Obstacle circulaire fixe contre lequel les particules entrent en collision.
- Chargement d'une scène depuis un fichier texte, avec une exception levée et rattrapée sur un fichier invalide.

Bonus : rendre `Vec2` générique (`Vec2<T>`) avec un template, ou ajouter un troisième intégrateur (Runge-Kutta 4).

Rapport (3 pages maximum)

- Le diagramme de classes final.
- Le tableau des notions du cours, complété avec le fichier et la ligne de code de chaque notion.
- Deux choix de conception justifiés, par exemple : pourquoi l'héritage virtuel, pourquoi la copie de `World` est interdite.
- Le graphique d'énergie Euler contre Verlet du jalon 2.
- Les limites connues du moteur.

Critères de validation

- ☐ L'extension est ajoutée sans modifier `World`, `Integrator` ni aucune force existante (vérifiable avec `git diff J3 J4`).
- ☐ Le rapport est déposé en PDF dans le dépôt.
- ☐ La démonstration enchaîne les quatre scènes, un changement d'intégrateur en direct et l'extension.
- ☐ Chaque membre de l'équipe sait expliquer n'importe quelle partie du code, pas seulement la sienne.

Tag Git : `J4`.

Répartition à trois et planning

Chaque membre possède un domaine du code, mais tous écrivent au moins une classe polymorphe. Les interfaces (`Force`, `Integrator`, `Drawable`, `Renderer`) sont écrites ensemble au début du jalon 2, avant de se séparer.

Jalon	Membre A — maths et intégration	Membre B — forces et scènes	Membre C — affichage et interaction	Durée
J1	`Vec2` et ses tests	`Particle`, rebond sur le sol	CMake, README, sortie CSV	2 h
J2	`Integrator`, Euler, Verlet, calcul d’énergie	`Force`, `Gravity`, `Spring`, `Drag`, scènes pendule et corde	`World`, règle des trois, graphique d’énergie	3 h
J3	`Entity`, diamant, mesures de `sizeof`	Scènes tissu et blob	`Renderer`, SFML, SVG, souris et clavier	4 h
J4	Diagramme de classes, partie conception du rapport	Extension choisie	Tableau des notions, préparation de la démo	2 h
Total				**11 h par personne**

Toute modification d’une interface commune passe par une relecture des deux autres membres avant fusion dans la branche principale.